

An Introduction to SQL

Chapter 4



What will you learn?

1. Why SQL?
2. Query
 - Basic Structure
 - Set Operations
 - Aggregate Functions
 - Null Values
 - Nested Subqueries
 - Views
3. Modification of the Database
4. Data Definition Language

1. Why SQL? ...

- SQL was initially developed by IBM round 1976.
- It has clear link to relational algebra.
- SQL includes both data definition operations and data manipulation operations.
 - Data definition language (DDL)
 - Data manipulation language (DML)
- There are many different SQL implementations; available in Oracle, DB2,SQL Server, Sybase, Access, JDBC etc. Most are a superset of a subset of one of the standards (ISO, ANSI, BSI ...)
- Learn SQL and can learn most databases.
- It can be embedded within other high-level programming languages such as C, Pascal and PL/I

Banking Example

branch (branch-name, branch-city, assets)

customer (customer-name, customer-street,
customer-city)

account (account-number, branch-name, balance)

loan (loan-number, branch-name, amount)

depositor (customer-name, account-number)

borrower (customer-name, loan-number)

Banking Example

branch		
branch-name	branch-city	assets
Downtown	Brooklyn	9000000
Redwood	Palo Alto	2100000
Perryridge	Horseneck	1700000
Mianus	Horseneck	400000
Round Hill	Horseneck	8000000
Pownal	Bennington	300000
North Town	Rye	3700000
Brighton	Brooklyn	7100000

account		
branch-name	account-number	balance
Downtown	A-101	500
Mianus	A-215	700
Perryridge	A-102	400
Round Hill	A-305	350
Brighton	A-201	900
Redwood	A-222	700
Brighton	A-217	750

borrow	
customer-name	loan-number
Jones	L-17
Smith	L-23
Hayes	L-15
Johnson	L-14
Curry	L-93
Smith	L-11
Williams	L-17
Adams	L-16

customer		
customer-name	customer-street	customer-city
Jones	Main	Harrison
Smith	North	Rye
Hayes	Main	Harrison
Curry	North	Rye
Lindsay	Park	Pittsfield
Turner	Putnam	Stamford
Williams	Nassau	Princeton
Adams	Spring	Pittsfield
Johnson	Alma	Palo Alto
Glenn	Sand Hill	Woodside
Brooks	Senator	Brooklyn
Green	Walnut	Stamford

loan		
branch-name	loan-number	amount
Downtown	L-17	1000
Redwood	L-23	2000
Perryridge	L-15	1500
Downtown	L-14	1500
Mianus	L-93	500
Round Hill	L-11	900
Perryridge	L-16	1300

depositor	
customer-name	account-number
Johnson	A-101
Smith	A-215
Hayes	A-102
Turner	A-305
Johnson	A-201
Jones	A-217
Lindsay	A-222

2. QUERY -basic structure

- A typical SQL query has the form:
select $A_1, A_2, \dots, A_n$
from $r_1, r_2, \dots, r_m$
where ρ ;
- The result of an SQL query is a relation.

2. Query-The select Clause

- The select clause corresponds to the **projection operation** of the relational algebra. It is used to list the attributes desired in the result of a query.
e.g. Find the names of all branches in the loan relation.

*select branch-name
from loan;*

In the "pure" relational algebra syntax, the query would be:
loan{branch-name}

- An asterisk in the select clause denotes "all attributes"

*select *
from loan;*

2. Query -The select Clause...

- SQL **allows duplicates** in relations as well as in query results. To force the elimination of duplicates, insert the keyword **distinct** after select.

e.g. Find the names of all branches in the loan relations, and remove duplicates

```
select distinct branch-name  
from loan;
```

- The keyword **all** specifies that duplicates not be removed.

```
select all branch-name  
from loan;
```

- The select clause can contain arithmetic expressions involving the operation **+**, **-**, *****, and **/**, and operating on constants or attributes of tuples.

e.g. *select branch-name, loan-number, **amount*100**
from loan;*

2. Query - The where Clause

- The **where** clause corresponds to the selection predicate of the relational algebra.

- **and, or, not**

e.g. Find all loan number for loans made by the Perryridge branch with loan amounts greater than \$1200.

select loan-number

from loan

where branch-name = "Perryridge" and amount > 1200;

- **between**

e.g. Find the loan number of those loans with loan amounts between \$90,000 and \$100,000 (that is, $\geq \$90,000$ and $\leq \$100,000$)

select loan-number

from loan

where amount between 90000 and 100000;

2. Query - The from Clause

- The from clause corresponds to the Cartesian product operation of the relational algebra. It lists the relations to be scanned in the evaluation of the expression.

e.g. Find the Cartesian product borrower JOIN loan

*select **

from borrower, loan ;

e.g. Find the name and loan number of all customers having a loan at the Perryridge branch.

Select distinct customer-name, borrower. loan-number

from borrower, loan

*where borrower.loan-number = loan.loan-number
and*

branch-name = "Perryridge" ;

2. Query - Rename Operation

- The SQL mechanism for renaming relations and attributes is accomplished through the **as clause**:

old-name as new-name

e.g. Find the name and loan number of all customers having a loan at the Perryridge branch; replace the column name loan-number with the name loan-id.

```
select distinct customer-name, borrower.loan-  
number as loan-id
```

```
from borrower, loan
```

```
where borrower.loan-number = loan.loan-number and  
branch-number = "Perryridge"
```

2. Query - Tuple Variables

- **Tuple variables** are defined in the from clause via the use of the **as clause**.

e.g. Find the customer names and their loan numbers for all customers having a loan at some branch.

```
select  distinct customer-name, T.loan-number  
from    borrower as T, loan as S  
where   T.loan-number = S.loan-number
```

e.g. Find the names of all branches that have greater assets than some branch located in Brooklyn.

```
select  distinct T.branch-name  
from    branch as T, branch as S  
Where   T.assets > S.assets and S.branch-city = "Brooklyn"  
?
```

branch as **T**

branch-name	branch-city	assets
Downtown	Brooklyn	9000000
Redwood	Palo Alto	2100000
Perryridge	Horseneck	1700000
Mianus	Horseneck	400000
Round Hill	Horseneck	8000000
Pownal	Bennington	300000
North Town	Rye	3700000
Brighton	Brooklyn	7100000

Times



branch as **S**

branch-name	branch-city	assets
Downtown	Brooklyn	9000000
Redwood	Palo Alto	2100000
Perryridge	Horseneck	1700000
Mianus	Horseneck	400000
Round Hill	Horseneck	8000000
Pownal	Bennington	300000
North Town	Rye	3700000
Brighton	Brooklyn	7100000

T.assets > S.assets and S.branch-city = "Brooklyn"

2. Query - String Operations

- SQL includes a string-matching operator for comparisons on character strings. Patterns are described using two special characters:
 - percent (%). The % character matches any substring.
 - underscore (_). The _ character matches any character.

e.g. Find the names of all customers whose street includes the substring "Main".

select customer-name

From customer

where customer-street like "%Main%";

e.g. Match the name "Main%"

like "Main\%" escape "\"

2. Query- Ordering the Display of Tuples

- List in alphabetic order the names of all customers having a loan in Perryridge branch

```
select distinct customer-name  
from borrower, loan  
where borrower loan-number = loan.loan-number  
and  
branch-name = "Perryridge"  
order by customer-name desc;
```

- We may specify **desc** for descending order or **asc** for ascending order, for each attribute; ascending order is the default.

2. Query - Set Operations

- The set operations **union**, **intersect**, and **minus** operate on relations correspond to the relational algebra operations union, intersect, and minus.
- Each of the above operations **automatically eliminates duplicates**.

e.g. Find all customers who have a loan, an account, or both:

(select customer-name from depositor)

Union

(select customer-name from borrower);

2. Query- Set Operations...

e.g. Find all customers who have both a loan and an account.

select customer-name from depositor)

Intersect

(select customer-name from borrower);

e.g. Find all customers who have an account but no loan.

(select customer-name from depositor)

Minus

(select customer-name from borrower);

2. Query - Aggregate Functions

- These functions operate on the multiset of values of a column of a relation, and return a value.
 - **Avg()**: average value
 - **min ()**: minimum value
 - **max ()**: maximum value
 - **sum ()**: sum of values
 - **count (*)**: number of values
 - **COUNT(DISTINCT column-name)**: Determine the number of unique non-null values in a column

2. Query- Aggregate Functions...

e.g. Find the average account balance at the Perryridge branch.

```
select avg (balance)
from account
where branch-name = "Perryridge" ;
```

e.g. Find the number of tuples in the customer relation.

```
select count (*)
from customer ; count(*)计算所有行，包括空值；
```

e.g. Find the number of depositors in the bank.

```
select count (distinct customer-name)
from depositor ;
```

当对一个包含NULL值的列使用COUNT函数时，这个列中的NULL值将会被去除掉。

2. Query-Aggregate Functions-Group By

e.g. Find the number of depositors for each branch.

```
select branch-name, count (distinct customer-name)  
from depositor, account  
where depositor.account-number = account.account-  
number  
group by branch-name
```

- **Note:** Attributes in select clause outside of aggregate functions must appear in group by list.

account (account-number, branch-name, balance)

depositor (customer-name, account-number)

2. Query-Aggregate Functions-Group By

account		
branch-name	account-number	balance
Downtown	A-101	500
Mianus	A-215	700
Perryridge	A-102	400
Round Hill	A-305	350
Brighton	A-201	900
Redwood	A-222	700
Brighton	A-217	750

depositor	
customer-name	account-number
Johnson	A-101
Smith	A-215
Hayes	A-102
Turner	A-305
Johnson	A-201
Lindsay	A-222
Jones	A-217



branch-name	customer-name
Brighton	Jones
Brighton	Johnson
Downtown	Johnson
Mianus	Smith
Perryridge	Hayes
Redwood	Lindsay
Round Hill	Turner

result

branch-name	count
Brighton	2
Downtown	1
Mianus	1
Perryridge	1
Redwood	1
Round Hill	1

2. Query-Aggregate Functions - Having Clause

e.g. Find the names of all branches where the average account balance is more than \$600.

*select branch-name, avg (balance)
from account
group by branch-name
having avg (balance) > 600 ;*

▪ **Note:** predicates in the having clause are applied after the formation of groups.

account		
branch-name	account-number	balance
Downtown	A-101	500
Mianus	A-215	700
Perryridge	A-102	400
Round Hill	A-305	350
Brighton	A-201	900
Redwood	A-222	700
Brighton	A-217	750

account	
branch-name	balance
Downtown	500
Mianus	700
Perryridge	400
Round Hill	350
Redwood	700
Brighton	900
Brighton	750

account	
branch-name	balance
Downtown	500
Mianus	700
Perryridge	400
Round Hill	350
Redwood	700
Brighton	825

EMP and DEPT Example

I need the sum
of all salaries
for workdepts beginning
with the letter D

```
SELECT      SUM(SALARY) AS SUM
FROM        EMPLOYEE
WHERE       WORKDEPT LIKE 'D%'
```



SUM

373020.00



EMP and DEPT Example

I need a listing of the salaries for all employees in the departments A00, B01, and C01. In addition, for these departments, I want the totals spent for salaries.

Group By



```
SELECT WORKDEPT, SALARY
FROM EMPLOYEE
WHERE WORKDEPT IN ('A00', 'B01', 'C01')
ORDER BY WORKDEPT
```

WORKDEPT	SALARY
A00	52750.00
A00	46500.00
A00	29250.00
B01	41250.00
C01	38250.00
C01	23800.00
C01	28420.00

```
SELECT WORKDEPT, SUM(SALARY) AS SUM
FROM EMPLOYEE
WHERE WORKDEPT IN ('A00', 'B01', 'C01')
GROUP BY WORKDEPT
ORDER BY WORKDEPT
```

WORKDEPT	SUM
A00	128500.00
B01	41250.00
C01	90470.00

EMP and DEPT Example

Now, I just want to see
departments with total spent
for salaries higher than 50000



GROUP BY, HAVING

```
SELECT  
FROM  
WHERE  
GROUP BY  
ORDER BY  
WORKDEPT, SUM(SALARY) AS SUM  
EMPLOYEE  
WORKDEPT IN ('A00', 'B01', 'C01')
```



WORKDEPT	SUM
A00	128500.00
B01	41250.00
C01	90470.00

```
SELECT  
FROM  
WHERE  
GROUP BY  
HAVING  
ORDER BY  
WORKDEPT, SUM(SALARY) AS SUM  
EMPLOYEE  
WORKDEPT IN ('A00', 'B01', 'C01')  
SUM(SALARY) > 50000
```



WORKDEPT	SUM
A00	128500.00
C01	90470.00

EMP and DEPT Example

```
SELECT DEP, JOB, AVG(SAL)
FROM EMPL
WHERE JOB <> 'M'
GROUP BY DEP, JOB
HAVING AVG(SAL) > 28000
ORDER BY 3 DESC
```



Conceptual Execution

BASE TABLE

JOB	SAL	DEP
S	31000	BLU
M	32000	RED
S	30000	BLU
C	27000	GRE
S	33000	GRE
M	31000	BLU
S	32000	RED
C	28000	GRE
S	30000	RED
M	33000	GRE
S	31000	RED
S	35000	GRE
C	27000	BLU
S	29000	RED
S	29000	BLU

WHERE

JOB	SAL	DEP
S	31000	BLU
S	30000	BLU
C	27000	GRE
S	33000	GRE
S	32000	RED
C	28000	GRE
S	30000	RED
S	31000	RED
S	35000	GRE
C	27000	BLU
S	29000	RED
S	29000	BLU

GROUP BY

JOB	SAL	DEP
C	27000	BLU
S	31000	BLU
S	29000	BLU
S	30000	BLU
C	27000	GRE
C	28000	GRE
S	33000	GRE
S	35000	GRE
S	32000	RED
S	30000	RED
S	31000	RED
S	29000	RED

Conceptual Execution

HAVING			SELECT			ORDER BY		
JOB	SAL	DEP	DEP	JOB	AVG(SAL)	DEP	JOB	AVG(SAL)
S	31000	BLU	BLU	S	30000	GRE	S	34000
S	29000	BLU				RED	S	30500
S	30000	BLU				BLU	S	30000
S	33000	GRE	GRE	S	34000			
S	35000	GRE						
S	32000	RED	RED	S	30500			
S	30000	RED						
S	31000	RED						
S	29000	RED						

Change AVG(SAL) to SUM(SAL)?

EMP and DEPT Example

Display the departments
with more than one employee



```
SELECT      WORKDEPT,  
            COUNT(*) AS NUMB  
FROM EMPLOYEE  
GROUP BY  WORKDEPT  
ORDER BY  NUMB, WORKDEPT
```



<u>WORKDEPT</u>	<u>NUMB</u>
B01	1
E01	1
A00	3
C01	3
E21	4
E11	5
D21	6
D11	9

```
SELECT      WORKDEPT,  
            COUNT(*) AS NUMB  
FROM EMPLOYEE  
GROUP BY  WORKDEPT  
HAVING    COUNT(*) > 1  
ORDER BY  NUMB, WORKDEPT
```



<u>WORKDEPT</u>	<u>NUMB</u>
A00	3
C01	3
E21	4
E11	5
D21	6
D11	9

Product and Ptype Example

Product

Pno	Pname	Tno	Price	Stock
A001	文件夹	1	20	500
A002	笔筒	1	15	300
A003	三色圆珠笔	1	10	1000
B001	宠物小精灵丛书	2	100	2000
B002	白雪公主	2	15	300
B003	迪斯尼套书	2	200	200
C001	小儿西裤	3	40	200
C002	小儿短裙	3	60	500

Ptype

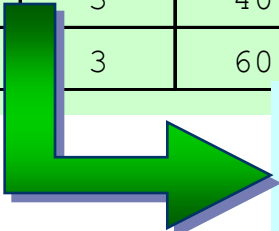
Tno	Tname
1	文具
2	儿童书籍
3	儿童服装

Product and Ptype Example

- Find the maximal price of each types .

```
SELECT Tno,MAX(Price)
FROM Product
GROUP BY Tno;
```

Pno	Pname	Tno	Price	Stocks	按Tno 相等的原则进行分组	
A001	文件夹	1	20	500	组1:Tno="1"	Tno Max(Price) 1 20
A002	笔筒	1	15	300		
A003	三色圆珠笔	1	10	1000		
B001	宠物小精灵丛书	2	100	2000	组1:Tno="2"	Tno Max(Price) 2 200
B002	白雪公主	2	15	300		
B003	迪斯尼套书	2	200	200		
C001	小儿西裤	3	40	200	组1:Tno="3"	Tno Max(Price) 3 60
C002	小儿短裙	3	60	500		

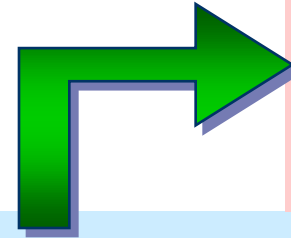


Tno	Max(Price)
1	20
2	200
3	60

Product and Ptype Example

- Find the number of all orders which ordered at least two products.

```
SELECT Ono  
FROM Order_item  
GROUP BY Ono  
HAVING COUNT(Pno)>=2;
```



Ono
0001
0002
0006
0008

Ono	Pno	Quantity
0001	A001	100
0001	A002	200
0002	A001	100
0002	A002	200
0002	A003	300
0003	B001	500
0004	B002	500
0005	B003	1000
0006	C001	200
0006	C002	300
0007	A001	200
0008	C001	300
0008	C002	500

挑选“Pno的个数大于等于2”的组的Ono



组1: Ono="0001"

0001

组2: Ono="0002"

0002

组3: Ono="0003"

组4: Ono="0004"

组5: Ono="0005"

组6: Ono="0006"

0006

组7: Ono="0007"

组8: Ono="0008"

0008

2. Query-Null Values

- It is possible for tuples to have a null value, denoted by null, for some of their attributes, null signifies an **unknown value** or that **a value does not exist**.
- The result of any arithmetic expression involving null is null.
- Roughly speaking, all comparisons involving null return false.
- More precisely, Any comparison with null returns unknown
 - (unknown or unknown) = unknown,
 - (true and unknown) = unknown,
 - (false and unknown) = false,
 - (unknown and unknown) = unknown
- **Note:** Result of where clause predicate is treated as false if it evaluates to unknown

2. Query-Null Values...

e.g. Find all loan number which appear in the loan relation with null values for amount.

```
select loan-number  
from loan  
where amount is null ;
```

- Total all loan amounts

```
select sum (amount)  
from loan ;
```

- **Note:** Above statement ignores null amounts; result is null if there is no non-null amount.
- **Note:** All aggregate operations except count(*) ignore tuples with null values on the aggregated attributes.

2. Query-Null Values...

SUM(SALARY)+SUM(BONUS)+SUM(COMM)

VERSUS

SUM(SALARY+BONUS+COMM)

<u>EMPNO</u>	<u>SALARY</u>	<u>BONUS</u>	<u>COMM</u>	<u>SALARY+BONUS+COMM</u>	
000010	1000	500	100	1600	
000020	2000	NULL	300	NULL	
000030	2500	400	NULL	NULL	
000040	1500	100	400	2000	
-----	-----	-----	----	-----	
Sum:	7000	1000	800	--> 8800	3600
	=====	=====	=====		=====

2. Query- Nested Subqueries

- SQL provides a mechanism for the nesting of subqueries.
- A subquery is a select-from-where expression that is nested within another query.
- A common use of subqueries is to perform tests for set membership and set comparisons.

2. Query- Nested Subqueries- Set Membership

$$F \text{ in } r \Leftrightarrow \exists t \in r (t = F)$$

(5 in

0
4
5

) = true

(5 in

0
4
6

) = false

(5 not in

0
4
6

) = true

2. Query- Nested Subqueries- Set Membership

e.g. Find all customers who have both an account and a loan at the bank.

```
select distinct customer-name  
from borrower  
where customer-name in (select customer-name  
from depositor);
```

e.g. Find all customers who have a loan at the bank but do not have an account at the bank

```
select distinct customer-name  
from borrower  
where customer-name not in (select customer-name  
from depositor);
```

2. Query- Nested Subqueries- Set Membership

e.g. Find all customers who have both an account and a loan at the North Town branch.

```
select distinct customer-name
from borrower, loan
where borrower.loan-number = loan.loan-number and
      branch-name = "North Town " and
      (branch-name, customer-name) in
      (select branch-name, customer-name
       from depositor, account
       where depositor.account-number =
             account.account-number);
```

```
account (account-number, branch-name, balance)
loan (loan-number, branch-name, amount)
depositor (customer-name, account-number)
borrower (customer-name, loan-number)
```

Subquery Using IN

List the people in Smith's department that have the same job as Smith



```
SELECT FIRSTNME, LASTNAME,  
       WORKDEPT, JOB  
FROM EMPLOYEE  
WHERE (WORKDEPT, JOB) IN  
      (SELECT WORKDEPT, JOB  
       FROM EMPLOYEE  
       WHERE LASTNAME = 'SMITH')
```

<u>FIRSTNME</u>	<u>LASTNAME</u>	<u>WORKDEPT</u>	<u>JOB</u>
JAMES	JEFFERSON	D21	CLERK
SALVATORE	MARINO	D21	CLERK
DANIEL	SMITH	D21	CLERK
SYBIL	JOHNSON	D21	CLERK
MARIA	PEREZ	D21	CLERK
ETHEL	SCHNEIDER	E11	OPERATOR
JOHN	PARKER	E11	OPERATOR
PHILIP	SMITH	E11	OPERATOR
MAUDE	SETRIGHT	E11	OPERATOR

Subquery Using NOT IN

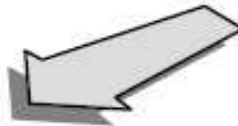
Which departments
do not have projects
assigned to them?



DEPARTMENT Table

<u>DEPTNO</u>	<u>DEPTNAME</u>
A00	SPIFFY COMPUTER SERVICE
B01	PLANNING
C01	INFORMATION CENTER
...	...

```
SELECT DEPTNO, DEPTNAME
FROM DEPARTMENT
WHERE DEPTNO NOT IN (SELECT DEPTNO
                      FROM PROJECT
                      WHERE DEPTNO IS NOT NULL)
```



Final Result

DEPTNO	DEPTNAME
A00	SPIFFY COMPUTER SERVICE



Result of Subquery

B01
C01
D01
D11
D21
E01
E11
E21

Subquery within HAVING Clause

I need a list of the departments whose average salary for non-managers is higher than the company-wide average for non-managers. The department with the highest average should be listed first.



```
SELECT  WORKDEPT, AVG(SALARY) AS AVG_WORKDEPT
FROM    EMPLOYEE
WHERE   JOB <> 'MANAGER'
GROUP BY WORKDEPT
HAVING  AVG(SALARY) > (SELECT AVG(SALARY)
                       FROM EMPLOYEE
                       WHERE JOB <> 'MANAGER')
ORDER BY AVG_WORKDEPT DESC
```

Final Result

WORKDEPT	AVG_WORKDEPT
A00	42833.33333333
C01	26110.00000000

Result of Subquery

25188.80000000

2. Query- Nested Subqueries- The Some Clause

(5 < some

0
5
6

) = true (read: 5 < some tuple in the relation)

(5 = some

0
5

) = true

(= some) $\equiv$ in

However, ($\neq$ some) $\neq$ not in

(5 $\neq$ some

0
5

) = true (since 0 $\neq$ 5)

2. Query- Nested Subqueries- Set Comparison

e.g. Find all branches that have greater assets than some branch located in Brooklyn.

```
select distinct T.branch-name
from branch as T, branch as S
where T.assets > S.assets and
      S.branch-city = "Brooklyn" and
      T.branch-city <> "Brooklyn";
```

branch as T			branch as S		
branch-name	branch-city	assets	branch-name	branch-city	assets
Downtown	Brooklyn	9000000	Downtown	Brooklyn	9000000
Redwood	Palo Alto	2100000	Redwood	Palo Alto	2100000
Perryridge	Horseneck	1700000	Perryridge	Horseneck	1700000
Mianus	Horseneck	400000	Mianus	Horseneck	400000
Round Hill	Horseneck	8000000	Round Hill	Horseneck	8000000
Pownal	Bennington	300000	Pownal	Bennington	300000
North Town	Rye	3700000	North Town	Rye	3700000
Brighton	Brooklyn	7100000	Brighton	Brooklyn	7100000

Times

T.assets > S.assets and S.branch-city = "Brooklyn"

2. Query- Nested Subqueries- The Some Clause

e.g. Find all branches that have greater assets than some branch located in Brooklyn.

```
select branch-name  
from branch  
where assets >some  
      (select assets  
       from branch  
       where branch-city = "Brooklyn" );  
??
```

2. Query- Nested Subqueries- The All Clause

(5 < all

0
5
6

) = false

(5 < all

6
10

) = true

(5 = all

4
5

) = false

(5 ≠ all

4
6

) = true (since 5 ≠ 4 and 5 ≠ 6)

2. Query- Nested Subqueries- The All Clause

e.g. Find the names of all branches that have greater assets than all branches located in Brooklyn.

```
select branch-name  
from branch  
where assets > all  
    (select assets  
     from branch  
     where branch-city =  
     "Brooklyn");
```

2. Query- Test for Empty Relations

- The exists construct returns the value true if the argument subquery is nonempty.

帶有EXISTS 和 NOT EXISTS谓词的查询的子查询不返回任何数据，只返回逻辑真和假.

exists $r \Leftrightarrow r \neq \emptyset$

not exists $r \Leftrightarrow r = \emptyset$

2. Query- Test for Empty Relations

查询所有选修了1号课程的学生姓名

Select Sname

from Student

where Exists

(select *

from SC

Where Sno=Student.Sno and Cno='1');

- ✓ 由exists引出的子查询，其目标列表表达式通常都用*
- ✓ 子查询的查询条件依赖于父查询的某个属性值（相关子查询）
- ✓ 反复求值
- ✓ 不能被其他形式的子查询等价替换

学生表: Student(Snumber, Sname, Ssex, Sage, Sdept)

课程表: Course(Cno, Cname, Cpno, Ccredit)

学生选课表: SC (Sno, Cno, Grade)

2. Query- Test for Empty Relations

查询没有选修了1号课程的学生姓名

Select Sname

from Student

where not Exists

(select *

from SC

Where Sno=Student.Sno and Cno='1');

学生表: Student(Snumber, Sname, Ssex, Sage, Sdept)

课程表: Course(Cno, Cname, Cpno, Ccredit)

学生选课表: SC (Sno, Cno, Grade)

2. Query- Test for Empty Relations

查询选修了全部课程的学生姓名

Select Sname

把全称量词转换成等价的使用存在量词的形式。

from Student

$\Rightarrow$ 没有一门课程是他不选修的。

where not exists

(select *

from Course

Sno不选的所有课

where not exists

(select *

from SC

where Sno=Student.Sno

Sno 不选的某个课

and Cno=Course.Cno));

学生表: Student(Snumber, Sname, Ssex, Sage, Sdept)

课程表: Course(Cno, Cname, Cpno, Ccredit)

学生选课表: SC (Sno, Cno, Grade)

Product and Ptype Example

Product

Pno	Pname	Tno	Price	Stock
A001	文件夹	1	20	500
A002	笔筒	1	15	300
A003	三色圆珠笔	1	10	1000
B001	宠物小精灵丛书	2	100	2000
B002	白雪公主	2	15	300
B003	迪斯尼套书	2	200	200
C001	小儿西裤	3	40	200
C002	小儿短裙	3	60	500

Ptype

Tno	Tname
1	文具
2	儿童书籍
3	儿童服装

Product and Ptype Example

- Find the number of all orders which ordered "A001" product.

```
SELECT DISTINCT Ono
FROM Order
WHERE EXISTS
  (SELECT *
   FROM Order_item
   WHERE Ono=Order.Ono
   AND
     Pno="A001");
```



Cno
C001
C002
C004

Product and Ptype Example

根据 Order 表当前元组的值，对 Order_item 表进行扫描，找到满足条件的元组就停止

①	Ono	Cno	Odate	Freight	②
→	0001	C001	1999-12-03	20	→
③	0002	C002	1999-12-10	50	
↓	0003	C001	2000-04-05	20	
	0004	C002	2000-04-20	20	
	0005	C003	1999-12-20	100	
	0006	C003	2000-06-09	50	
	0007	C004	2000-07-08	50	
	0008	C005	2000-10-08	100	

Ono	Pno	Quantity
0001	A001	100
0001	A002	200
0002	A001	100
0002	A002	200
0002	A003	300
0003	B001	500
0004	B002	500
0005	B003	1000
0006	C001	200
0006	C002	300
0007	A001	200
0008	C001	300
0008	C002	500

Product and Ptype Example

- Find the number of all orders which didn't order "A001" product.

```
SELECT DISTINCT Ono
FROM Order
WHERE NOT EXISTS
  (SELECT *
   FROM Order_item
   WHERE Ono=Order.Ono AND
        Pno="A001");
```

2. Query- Test for Empty Relations

e.g. Find all customers who have an account at all branches located in Brooklyn.

```
select distinct S.customer-name
from depositor as S
where not exists (
    (select branch-name
     from branch // Find branch-name located in "Brooklyn")
     where branch-city = "Brooklyn")
minus
    (select R.branch-name
     from depositor as T, account as R
     where T.account-number = R.account-number and
           S.customer-name = T.customer-
name); // Find S.customer-name's branch-name
```

Note: $X \text{ minus } Y = \emptyset \Leftrightarrow X \subseteq Y$

3. Modification– Deletion

e.g. Delete all account records at the Perryridge branch.

```
delete from account  
where branch-name = "Perryridge" ;
```

e.g. Delete all accounts at every branch located in Needham.

```
delete from depositor          //The order is important  
where account-number in  
    (select account-number  
     from branch, account  
     where branch-city = "Needham"  
       and branch.branch-name = account.branch-name) ;  
  
delete from account  
where branch-name in (select branch-name  
                     from branch  
                     where branch-city = "Needham") ;
```

3. Modification– Deletion

e.g. Delete the record of all accounts with balances below the average at the bank.

```
delete from account  
where balance < (select avg (balance)  
                 from account);
```

Problem: as we delete tuples from deposit, the average balance changes

Solutions used in SQL:

- 1.First, compute avg balance and find all tuples to delete
- 2.Next, delete all tuples found above (without recomputing avg or retesting the tuples)

3. Modification– Insertion

e.g. Add a new tuple to account.

```
insert into account  
values (A-9732 , "Perryridge", 1200) ;
```

```
insert into account (branch-name, balance, account-  
number)  
values ("Perryridge", 1200, A-9732) ;
```

e.g. Add a new tuple to account with balance set to null.

```
insert into account  
values (A-777, "Perryridge", null)
```

3. Modification– Insertion

e.g. Provide as a gift for all loan customers of the Perryridge branch, a \$200 savings account. Let the loan number serve as the account number for the new savings account

insert into account

```
select loan-number, branch-name, 200
from loan
where branch-name = "Perryridge" ;
```

insert into depositor

```
select customer-name, loan-number
from loan, borrower
where branch-name = "Perryridge" and
      loan.loan-number = borrower.loan-number ;
```

3. Modification – Updates

e.g Increase all accounts with balances over \$10,000 by 6%, all other accounts receive 5%.

Write two **update** statements:

```
update account  
set balance = balance * 1.06  
where balance > 10000 ;
```

```
update account  
set balance = balance * 1.05  
where balance <= 10000 ;
```

NOTE: The order is important.

4. Data Definition -Create Table

- An SQL relation is defined using the create table command:

```
create table R (A1 D1, A2 D2, ..., An Dn,  
    (integrity-constraint1),..., (integrity-constraintk)) ;
```

- R is the name of the relation
- each A_i is an attribute name in the schema of relation R
- D_i is the data type of values in the domain of attribute A_i

e.g.

```
create table branch  
    (branch-name char(15) not null,  
    branch-city char(30),  
    assets integer) ;
```

4. Data Definition -Create Table Integrity Constraints

- not null
 - primary key ($A_1, \dots, A_n$)
 - check (P), where P is a predicate
- e.g. Declare branch-name as the primary key for branch and ensure that the values of assets are non-negative.

create table branch

```
(branch-name char(15),  
branch-city char(30),  
assets integer,  
primary key (branch-name),  
check (assets >= 0));
```

NOTE: primary key declaration on an attribute automatically ensures not null in SQL-92

4. Data Definition - Drop and Alter Table

- The drop table command deletes all information about the dropped relation from the database.

`drop table emp`

- The alter table command is used to add attributes to an existing relation.

`alter table R add A D`

- A is the name of the attribute to be added to relation R and D is the domain of A.
- All tuples in the relation are assigned null as the value for the new attribute.
- The alter table command can also be used to drop attributes of a relation

`alter table R drop A`

- A is the name of an attribute of relation R

4. Data Definition - Views

- Provide a mechanism to hide certain data from the view of certain users.

create view v as <query expression>

- <query expression> is any legal expression
- The view name is represented by v

4. Data Definition - Views

- Create a view consisting of branches and their customers

create view all-customer as

*(select branch-name, customer-name
from depositor, account
where depositor account-number = account.account-number)*

union

*(select branch-name, customer-name
from borrower, loan
where borrower.loan-number = loan.loan-number) ;*

- Find all customers of the Perryridge branch

*select customer-name
from all-customer
where branch-name = "Perryridge"*

4. Data Definition - Views

- Create a view of all loan data in *loan* relation, hiding the *amount* attribute

```
create view branch-loan as  
    select branch-name, loan-number  
    from loan ;
```

- Add a new tuple to *branch-loan*

```
insert into branch-loan  
    values ("Perryridge", "L-307") ;
```



```
insert into loan  
    values("Perryridge", "L-307, null) ;
```

- Updates on more complex views are difficult or impossible to translate, and hence are disallowed.

4. Data Definition - Views

DB2不允许更新视图规定:

- 1 视图由两个以上基本表导出
- 2 视图字段来自字段表达式或常数, 不可执行insert和update, 但允许执行delete
- 3 视图字段来自聚集函数
- 4 视图中含group by
- 5 视图中含distinct
- 6 视图定义中含有嵌套查询;
- 7 不允许更新的视图上定义的视图

More Examples

```
CREATE TABLE S
( S#      CHAR(5),
  SNAME    CHAR(20),
  STATUS   NUMERIC(5),
  CITY     CHAR(15),
  PRIMARY KEY ( S# ) ) ;

CREATE TABLE P
( P#      CHAR(6),
  PNAME    CHAR(20),
  COLOR    CHAR(6),
  WEIGHT   NUMERIC(5,1),
  CITY     CHAR(15),
  PRIMARY KEY ( P# ) ) ;

CREATE TABLE SP
( S#      CHAR(5),
  P#      CHAR(6),
  QTY     NUMERIC(9),
  PRIMARY KEY ( S#, P# ),
  FOREIGN KEY ( S# ) REFERENCES S,
  FOREIGN KEY ( P# ) REFERENCES P ;
```

More Examples

S

S#	SNAME	STATUS	CITY
S1	Smith	20	London
S2	Jones	10	Paris
S3	Blake	30	Paris
S4	Clark	20	London
S5	Adams	30	Athens

P

P#	PNAME	COLOR	WEIGHT	CITY
P1	Nut	Red	12.0	London
P2	Bolt	Green	17.0	Paris
P3	Screw	Blue	17.0	Rome
P4	Screw	Red	14.0	London
P5	Cam	Blue	12.0	Paris
P6	Cog	Red	19.0	London

SP

S#	P#	QTY
S1	P1	300
S1	P2	200
S1	P3	400
S1	P4	200
S1	P5	100
S1	P6	100
S2	P1	300
S2	P2	400
S3	P2	200
S4	P2	200
S4	P4	300
S4	P5	400

More Examples

- 7.7.1 Get color and city for "nonParis" parts with weight greater than ten pounds.

```
SELECT PX.COLOR, PX.CITY  
FROM   P AS PX  
WHERE  PX.CITY <> 'Paris'  
AND    PX.WEIGHT > 10.0 ;
```

- 7.7.2 For all parts, get the part number and the weight of that part in grams.

```
SELECT P.P#, P.WEIGHT * 454 AS GMWT  
FROM   P ;
```

- 7.7.3 Get all combinations of supplier and part information such that the supplier and part in question are colocated.

```
SELECT S.*, P.P#, P.PNAME, P.COLOR, P.WEIGHT  
FROM   S, P  
WHERE  S.CITY = P.CITY ;
```

More Examples

- 7.7.4 Get all pairs of city names such that a supplier located in the first city supplies a part stored in the second city.

```
select distinct s.city as scity, p.city as pcity  
from s,sp,p  
where s.s#=sp.s# and sp.p#=p.p#
```

- Note:** The following is not correct (because it includes CITY as a joining column in the second join):

```
SELECT DISTINCT S.CITY AS SCITY, P.CITY AS PCITY  
FROM      S NATURAL JOIN SP NATURAL JOIN P ;
```

More Examples

- 7.7.5 Get all pairs of supplier numbers such that the two suppliers concerned are colocated.

```
SELECT A.S# AS SA, B.S# AS SB  
FROM   S AS A, S AS B  
WHERE  A.CITY = B.CITY  
AND    A.S# < B.S# ;
```

- 7.7.6 Get the total number of suppliers.

```
SELECT COUNT(*) AS N  
FROM   S ;
```

More Examples

- 7.7.7 Get the maximum and minimum quantity for part P2.

```
SELECT MAX ( SP.QTY ) AS MAXQ, MIN ( SP.QTY ) AS MINQ  
FROM    SP  
WHERE   SP.P# = 'P2' ;
```

- 7.7.8 For each part supplied, get the part number and the total shipment quantity.

```
SELECT SP.P#, SUM ( SP.QTY ) AS TOTQTY  
FROM    SP  
GROUP   BY SP.P# ;
```

More Examples

- 7.7.9 Get part numbers for parts supplied by more than one supplier.

```
SELECT SP.P#  
FROM   SP  
GROUP  BY SP.P#  
HAVING COUNT ( SP.S# ) > 1 ;
```

- 7.7.10 Get supplier names for suppliers who supply part P2.

```
SELECT DISTINCT S.SNAME  
FROM   S  
WHERE  S.S# IN  
      ( SELECT SP.S#  
        FROM   SP  
        WHERE  SP.P# = 'P2' ) ;
```

More Examples

- 7.7.11 Get supplier names for suppliers who supply at least one red part.

```
SELECT DISTINCT S.SNAME
FROM S
WHERE S.S# IN
      ( SELECT SP.S#
        FROM SP
        WHERE SP.P# IN
              ( SELECT P.P#
                FROM P
                WHERE P.COLOR = 'Red' ) ) ;
```

More Examples

- 7.7.12 Get supplier numbers for suppliers with status less than the current maximum status in the S table.

```
SELECT S.S#  
FROM S  
WHERE S.STATUS <  
      ( SELECT MAX ( S.STATUS )  
        FROM S ) ;
```

- 7.7.13 Get supplier names for suppliers who supply part P2.

```
SELECT DISTINCT S.SNAME  
FROM S  
WHERE EXISTS  
      ( SELECT *  
        FROM SP  
        WHERE SP.S# = S.S#  
        AND SP.P# = 'P2' ) ;
```

More Examples

- 7.7.14 Get supplier names for suppliers who do not supply part P2.

```
SELECT DISTINCT S.SNAME
FROM S
WHERE NOT EXISTS
( SELECT *
  FROM SP
  WHERE SP.S# = S.S#
        AND SP.P# = 'P2' ) ;
```

Alternatively:

```
SELECT DISTINCT S.SNAME
FROM S
WHERE S.S# NOT IN
( SELECT SP.S#
  FROM SP
  WHERE SP.P# = 'P2' ) ;
```

More Examples

- 7.7.15 Get supplier names for suppliers who supply all parts.

```
SELECT DISTINCT S.SNAME
FROM   S
WHERE  NOT EXISTS
      ( SELECT *
        FROM   P
        WHERE  NOT EXISTS
              ( SELECT *
                FROM   SP
                WHERE  SP.S# = S.S#
                  AND  SP.P# = P.P# ) ) ;
```

More Examples

- 7.7.16 Get part numbers for parts that either weigh more than 16 pounds or are supplied by supplier S2, or both.

```
SELECT P.P#  
FROM P  
WHERE P.WEIGHT > 16.0  
  
UNION  
  
SELECT SP.P#  
FROM SP  
WHERE SP.S# = 'S2' ;
```

Group project—section 3

- Analyze the common query requirements (operations) based on your application background.
- Do you need some views in your background? why? What is their functions?
- Express them in SQL language.